

Sai Putumbaka

AI Resume and Job Application Strategist – Milestone II Design Document

CIS 4394 – Agentic AI

Overview

The AI Resume and Job Application Strategist is a multi-agent system designed to help users tailor job application materials more effectively. The system focuses on a common problem faced by students, early-career professionals, and career changers: manually adapting resumes and related materials for individual job descriptions is time-consuming and difficult to do consistently. To address this, I implemented a multi-agent workflow that accepts resume text, a target job description, and a requested output type, then generates structured application support. The system currently supports three primary outputs: tailored resume suggestions, cover letter generation, and application review. The goal of the design is to separate the workflow into specialized agents so that planning, generation, and evaluation are handled by different components rather than a single monolithic prompt.

Agent Roles and Workflow

The system uses a fixed sequential multi-agent workflow implemented with Google's Agent Development Kit (ADK). The coordination mechanism is a SequentialAgent, which executes a defined sequence of agents in a predictable order. The first agent is the Planner Agent. Its role is to interpret the user's task, identify the requested output type, extract key requirements from the job description, and create a concise task plan for the next stage. The second agent is the Executor Agent. Its role is to generate the main content requested by the user, whether that is tailored resume content, a cover letter, or an application review. The executor is instructed to remain grounded in the provided resume text and avoid inventing skills, achievements, or experiences. The third agent is the Reviewer Agent. Its role is to assess the output for relevance, clarity, factual grounding, and ATS alignment. The reviewer also structures the final output into clearly labeled sections so the response is easier for the user to read and use.

Communication and Data Flow

The workflow begins in the Streamlit front end, where the user pastes resume text, pastes a job description, and chooses the desired output type. These inputs are passed to an ADK bridge layer that creates a runtime message for the multi-agent system. The bridge uses the ADK Runner and InMemorySessionService to execute the workflow in a managed session. The prompt is sent into the root agent, which is the sequential workflow composed of the

planner, executor, and reviewer agents. Each agent receives the evolving context from the previous stage, allowing the workflow to progress from task interpretation to content generation to evaluation. A simple tool function is also included for keyword extraction from the job description. This tool supports the system by identifying important terms that can guide job alignment. Once the sequential workflow completes, the final response is returned to the Streamlit interface, where it is displayed to the user and can also be downloaded as a text file.

Failure Handling and Quality Control

Several measures were included to improve reliability and output quality. First, the Executor Agent is explicitly instructed not to hallucinate or invent unsupported experiences. This helps reduce the risk of generating misleading application content. Second, the Reviewer Agent functions as a quality-control layer by checking alignment, professionalism, and overall readiness. Third, because the system is running on the free Gemini tier, the ADK bridge includes a throttling delay between event steps to reduce the likelihood of rate-limit errors. The bridge also catches quota-related and other runtime exceptions and returns readable fallback messages instead of crashing the Streamlit application. This improves the stability of the overall system and ensures the user receives understandable feedback even when the model backend is temporarily limited.

Front-End Integration

The multi-agent backend is connected to a polished Streamlit front end designed to look more like a real career-support application than a basic technical demo. The interface includes a branded header, workflow summary, styled input panels, and a structured output section. The front end allows the user to enter resume text, paste a job description, select an output type, and run the workflow with a single button click. Although the front end is separate from ADK Web, the backend agents are also organized into an ADK-native package structure so they can be tested independently in the ADK development interface. This hybrid architecture combines the strengths of both approaches: a polished custom user experience through Streamlit and a clean agent-development workflow through ADK. This design choice makes the system both usable for end users and easier to debug and refine during development.